Problem Set #1 (90 points) **Due date:** September 13, 2024, 11:59 PM

REMINDER: As you work on your coding assignments, it is important to remember that passing the examples provided does not guarantee full credit. While these examples can serve as a helpful starting point, it is ultimately your responsibility to create additional tests, ensure that it is functioning correctly and meets all the requirements and specifications outlined in the assignment instructions. Your grade for this assignment is divided in two parts:

50% > Passing all post-due date tests
(no partial credit for failed cases)

50% > Clarity and design of your code
(Class Coding Standards, requirements/specifications)

Before you start…

- You are not allowed to use material that has not been covered in class.
- Your code must be written using Python language.
- Function names must be defined as stated in the function description. Failure to comply with this requirement will result in a 0 score for the post-due date testing.
- No code should be written outside functions, including function calls. All testing code must be done inside the `main` function
- Your functions should NOT have any `input()` calls in its body.
- Your code should be concise and efficient without useless code or redundant code.
- Failure to follow the assignment's requirements will result in a zero score for the assignment even if the autograder gives you full score.
- Code submitted with syntax errors does not receive ANY credit.
- Revise the Grading Notes provided in the Canvas Assignment that includes the evaluation criteria for Clarity and Design.
- You will be writing several functions, but they will all be saved in one file: PS1.py. Please save all the functions in this one file or you will lose points.
- Ask questions using our Problem Set 1 channel in Microsoft Teams.

***** For all functions below, you may only use sequence statements (variables, constants, expressions, and assignment/print/return statements) in your solution. *****

Part 1: Mathematical Expressions

Convert the following mathematical expressions into functions. Use the name indicated on the left-hand side as the name of your function. You may assume that all inputs to these functions are integers and that the output is a float. For this part, there is no need to provide a docstring, but you must provide annotations in the function definition.

Function #1 (5 pts)

$$math\_func\_1(x) = \frac{10x - 3}{8} \times \frac{7}{4x - 3}$$

Function #2 (5 pts)

$$math\_func\_2(a, b, c) = \frac{2\%a - 3}{c \times b} + (4//c) - b$$

Part 2: Conversions

Problem 2.1 (10 pts) Define and implement a function named exactly `m_to_arshins`. Your function should take in a number in meters and coverts that value into arshins. For reference, there are 1.406 arshins in 1 meter. Return the result of the conversion.

Examples:

```
>>> m_to_arshins(3)
4.218
>>> m_to_arshins(6)
8.436
```

Problem 2.2 (10 pts) Define and implement a function named exactly `convert_min`. Your function should take exactly one argument, a positive integer that represents the amount of minutes, and displays a string with the number of years and days for the minutes. The string must match all characters as shown in the examples. After the function displays the message, the function returns the value of 1. You can assume a year has 365 days.

Examples:

```
>>> convert_min(129)
129 min = 0 year(s) and 0 day(s)
1
>>> convert_min(36548765)
36548765 min = 69 year(s) and 196 day(s)
1
```

Part 3: Find me the value

Problem 3.1 (10 pts) Define and implement a function named exactly `calc_final_price` that accepts two numerical arguments: the original price of an item, and the tax rate applied to that item. The original price is a number that is greater than 0, and the tax rate is an integer on the interval [0, 100] (e.g., a tax rate of 6 corresponds to a 6% tax rate, etc.). If the provided arguments do not fall in these ranges, the output of the function is undefined (i.e., no need to error check--whatever happens, happens). The function should return the final price of the item after accounting for taxes (initial price + taxes) as a float.

Examples:

```
>>> calc_final_price(35, 8)
37.8
>>> calc_final_price(150, 0)
150.0
>>> calc_final_price(58.69, 6)
62.2114
```

Problem 3.2 (10 pts) In physics, the displacement of a moving body represents its change in position over time while accelerating. Given initial velocity v_0 in m/s, acceleration `a` in m/s^2, and elapsed time `t` in s, the displacement of the body is:

$$s = v_0 t + \frac{1}{2} a t^2$$

Define and implement a function named exactly `get_displacement` that accepts three arguments, v_0 , `a` and `t` and returns the change in position as a float.

Examples:

```
>>> get_displacement(3, 4, 5)
65.0
>>> get_displacement(25, 3, 4)
124.0
>>> get_displacement(35.5, 3.5, 7)
334.25
```

Problem 3.3 (10 pts) Define and implement a function named exactly `p_volume`, that accepts two arguments, the first is a number that represents the height of a square pyramid and the second represents the length of the sides of the square base. The function returns the pyramid's volume as a float. -- *Note*: To find the volume of a pyramid, we multiply the area of its base with the height of the pyramid, and divide by 3.

Examples:

```
>>> p_volume(12,6)
144.0
>>> p_volume(15,6.5)
211.25
>>> p_volume(45.5,2.75)
114.69791666666667
```

Part 4: Heat Index

The Weather Service has a formula to calculate precisely how hot it will be during the day, we call this value the *heat index*, which provides the apparent temperature given the atmospheric temperature and relative humidity. For this last part of the assignment, you will implement a series of functions that will help you calculate and display the apparent temperature so you know what to expect when you head outdoors.

Problem 4.1 (10 pts): Calculating the heat index is more of an estimation than anything else. Through experimental analysis, the National Oceanic and Atmospheric Administration has found a regression formula that roughly matches the experimental data:

$$heat\ index(T,H) = C_1 + C_2T + C_3H + C_4TH + C_5T^2 + C_6H^2 + C_7T^2H + C_8TH^2 + C_9T^2H^2$$

where:

- `T` is the temperature in Fahrenheit
- `H` is the humidity as a percentage in the range 0.0 to 100.0.
- `C1` is -42.379
- `C2` is 2.04901523
- `C3` is 10.14333127
- `C4` is -0.22475541
- `C5` is -6.83783×10^{-3}
- `C6` is -5.481717×10^{-2}
- `C7` is 1.22874×10^{-3}
- `C8` is 8.5282×10^{-4}
- `C9` is -1.99×10^{-6}

Define and implement a function named exactly `heat_index`, that takes a temperature in Fahrenheit and a number for humidity in the range of 0.0 to 100.0, and computes the equation above. Don't forget to avoid "magic-numbers", declare constants in the global scope for each of the C_n values.

Examples:

```
>>> heat_index(97.5, 46)
108.19859553750011
>>> heat_index(80, 97.3)
86.75719014570024
```

Problem 4.2 (10 pts): Given that floating-point numbers are an approximation anyway, we should devise a way to round these numbers to two decimal places, as that will be plenty accurate for our purposes. Define and implement a function named exactly `round_to_two`, that takes a decimal number and returns that number rounded to only two decimal places. Your function should round using the "Half Round-Up" method, so a number such as 4.2568 would round to 4.0 for zero decimal places, 4.3 for one, 4.26 for two, and so on.

⛔ To receive any credit for this problem, you are not allowed to use the Python built-in round() function!

💡 **Hint:**

The modulus operator and the floor division operator are handy tools when working with values that are better split into their remainder and whole divisor portions. While it might be mathematically silly to divide or modulate by one when using integers, this is a very helpful technique when working with floats!

`123.456 // 1.0` evaluates to `123.0`
`123.456 % 1.0` evaluates to `0.456`

In Module 2 lectures, we presented the technique to round for zero decimal places. If we want to use the same technique to round to one decimal place, we need to first shift the decimal point one place to the right (`value * 10`), apply the technique to round, and shift the decimal point back to the left (`value = value / 10`) to get the desired result. This approach can be extended to round to two decimal places!

Examples:

```
>>> round_to_two(85.6397)
85.64
>>> round_to_two(98.75)
98.75
>>> round_to_two(8.1)
8.1
```

Problem 4.3 (10 pts): Now that you have all the required components, define and implement a function named exactly design `get_apparent_temperature`, which takes the same parameters as the `heat_index` function in Problem 4.1. This function should call the appropriate functions to get the heat index and then display a message that tells the user the input temperature, humidity, and the resulting apparent temperature with the exact format as shown in the examples. You should round the values to two decimal places before displaying them. Afterward, this function should return the unrounded apparent temperature.

Examples:

```
>>> get_apparent_temperature(85.6397, 80.7306)
85.64 F and 80.73 % humidity feels like 99.06 F
99.06330624630884
>>> get_apparent_temperature(76, 98.75)
76.0 F and 98.75 % humidity feels like 74.97 F
74.965216759375
>>> get_apparent_temperature(83, 48)
83.0 F and 48.0 % humidity feels like 83.55 F
83.54963914000008
```